

Software Design Specification Template

Software Design Specification for <Project> based on event driven (function oriented) technology

Table of Contents

1. Introduction

1.1	Purpose
1.2	Document Conventions
1.3	Intended Audience and Reading Suggestions
1.4	References

2. Analysis Model

- 12.1 Methodology Used (If Structural)
- 12.2 Use Case diagram (Also include use case specifications)
- 12.3 ER Model
- 12.4 Data Flow Diagram (Also write Process Specifications)
- 12.5 Control Flow Diagram (Also write Control Specifications)
- 12.6 State Transition Diagram

3. Design Model

- 13.1 Architectural Design
 - 13.1.1 System Architecture Diagram
 - 13.1.2 Description of Architectural Design
- 13.2 Database Design
 - 13.2.1 Data Dictionary
 - 13.2.2 Normalization
- 13.3 Component Design
 - 13.3.1 Flow Chart
- 13.4 Interface Design
 - 13.4.1 Screenshots

Appendix B1 : Glossary

Appendix B2 : To Be Determined List

Introduction

1.1 Purpose

This document specifies the design and structure of the *Invisible Cloak Project*, a computer vision-based application that creates a real-time invisibility effect by detecting a specific color and replacing it with a captured background. The purpose of this document is to provide a clear understanding of system design, architecture, and implementation.

1.2 Document Conventions

- Headings and subheadings are used for clarity.
- Technical terms such as HSV, masking, and segmentation are used consistently.
- Requirements and processes are described in a structured format for easy understanding.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Students and developers implementing the system
- Faculty members evaluating the project
- Beginners learning computer vision concepts

Readers should start from the introduction, then proceed to the analysis model and design model.

1.4 References

1. Bradski, G., & Kaehler, A. (2008). [*Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.](#)
2. Kaehler, A., & Bradski, G. (2016). [*Learning OpenCV 3*. O'Reilly Media.](#)
3. Szeliski, R. (2010). [*Computer Vision: Algorithms and Applications*. Springer.](#)
4. Python Software Foundation. [Python Documentation. https://www.python.org](https://www.python.org)

2. Analysis Model

2.1 Methodology used

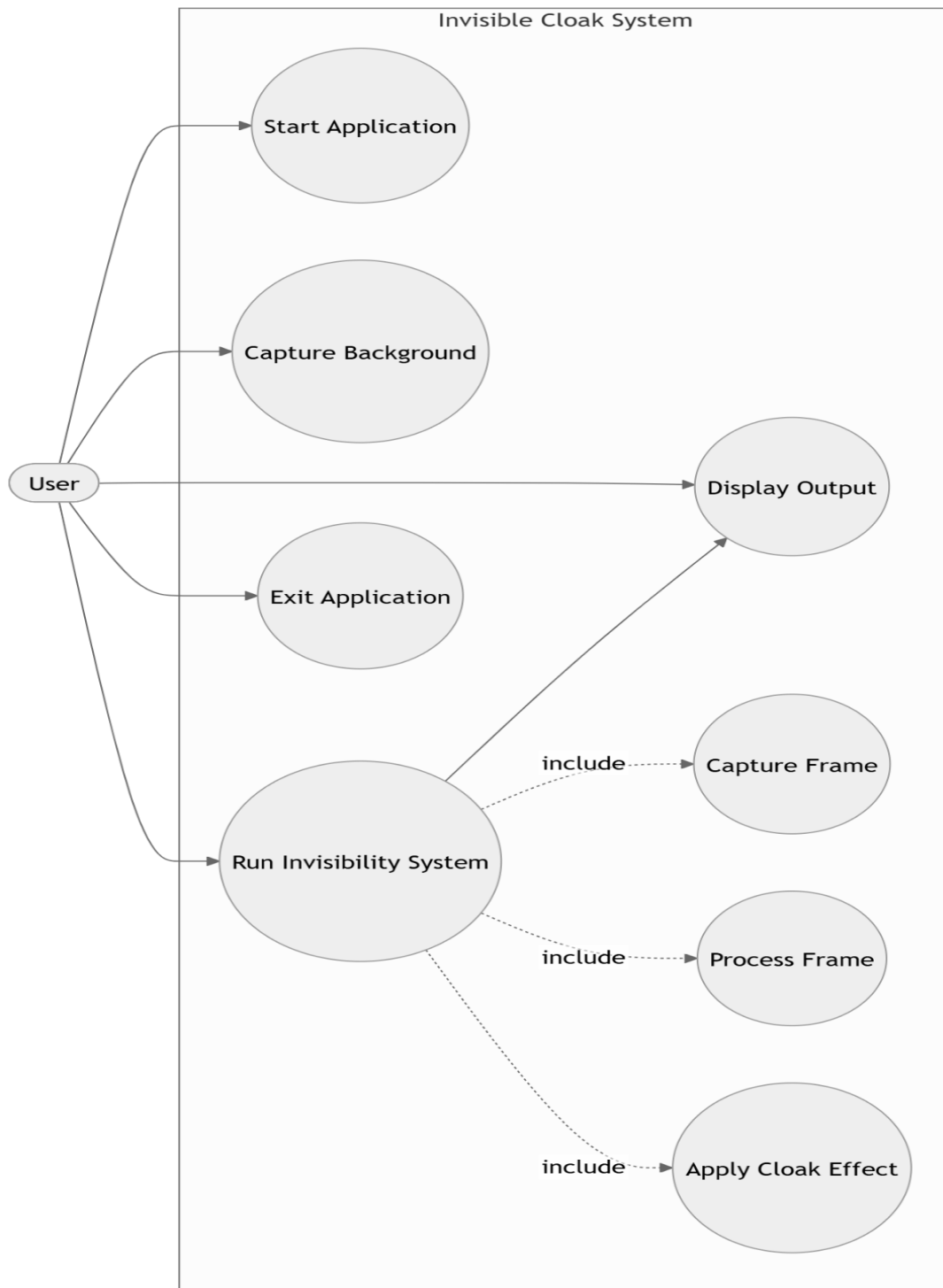
The system follows a step-by-step process to achieve the invisibility effect:

1. The webcam captures real-time video frames.
2. A background frame is captured initially when no object is present.
3. Each video frame is converted from BGR to HSV color space.
4. The system detects the specified color (pink) using predefined HSV ranges.
5. A mask is generated to isolate the detected region.

6. Morphological operations are applied to reduce noise.
7. The masked region is replaced with the stored background frame.
8. The final output is displayed in real-time.

This methodology ensures smooth processing and accurate invisibility effect.

2.2 Use Case diagram



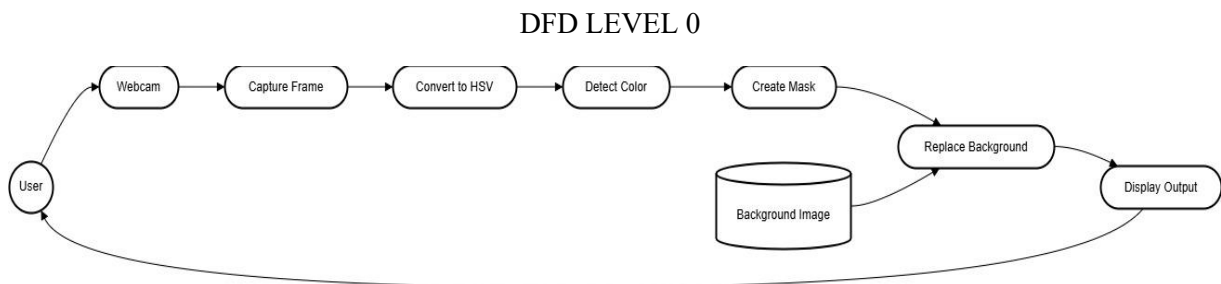
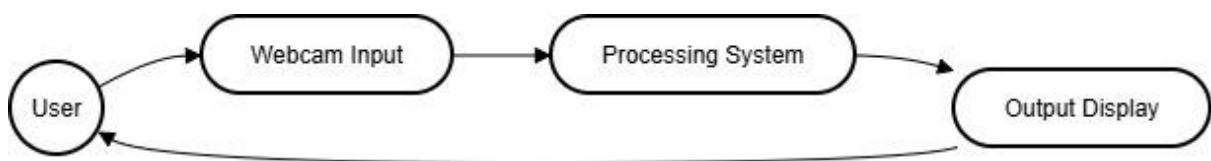
Use Case Specification

Name Of Use Case	Generate Invisibility Effect
Actor	User
Pre Condition	Camera is working User has cloak of specified color
Primary Flow of Events	User runs program System captures background System detects cloak color System replaces cloak area with background Output displayed
Alternate Flow Of Events	If camera fails → program stops If color not detected → no invisibility
Post Condition	Invisibility effect displayed

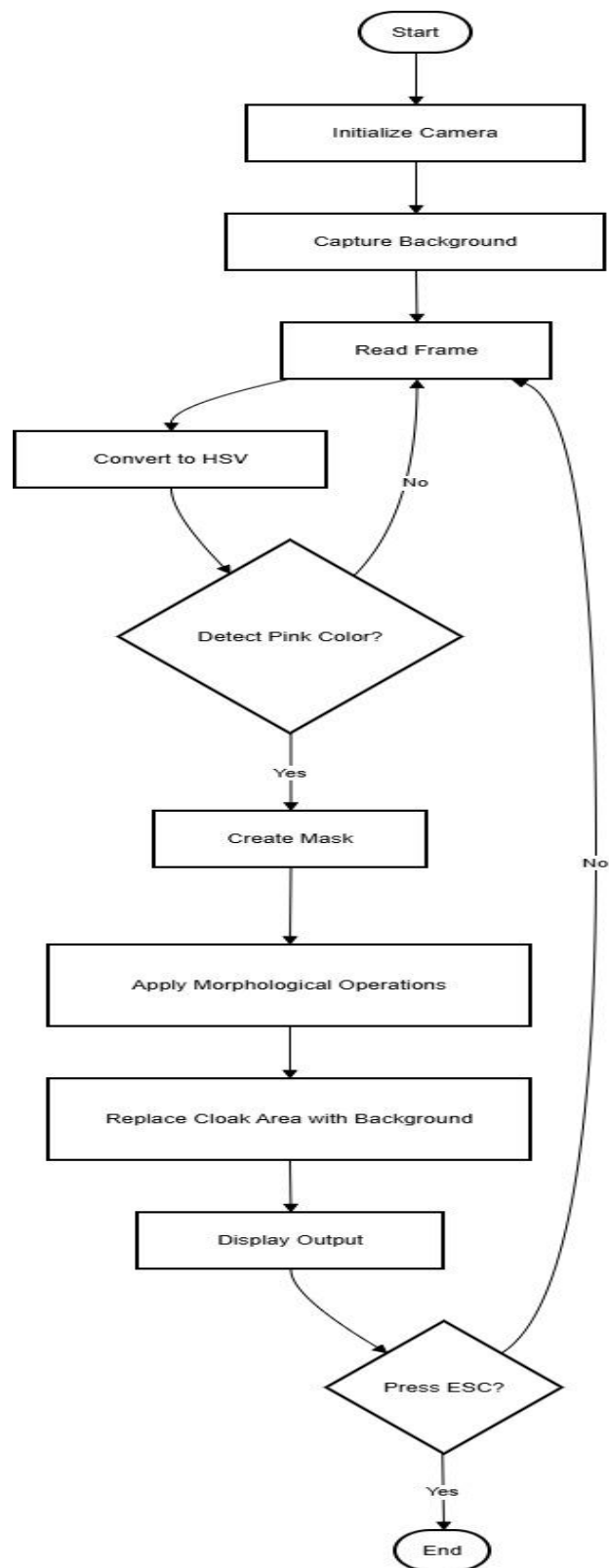
2.3 ER Model

No Database

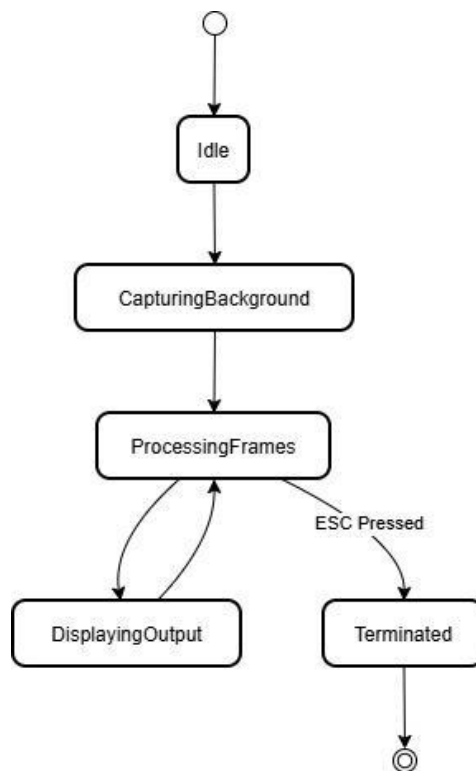
2.4 Data flow Diagram



2.5 Control Flow Diagram



2.6 State Transition Diagram



3. Design Model

3.1 Architectural Design

3.1.1 System Architectural Diagram

The system follows a **pipeline architecture**:

Input → Processing → Output

3.1.2 Description of Architectural Diagram

Input Module: Captures video

Processing Module: Applies computer vision techniques

Output Module: Displays result

- **Webcam:** Captures live video input
- **Python:** Controls program execution
- **OpenCV:** Handles image processing tasks
- **Output Window:** Displays final result

All components work together in real-time to produce the invisibility effect.

3.2 Database Design

3.2.1 Data Dictionary

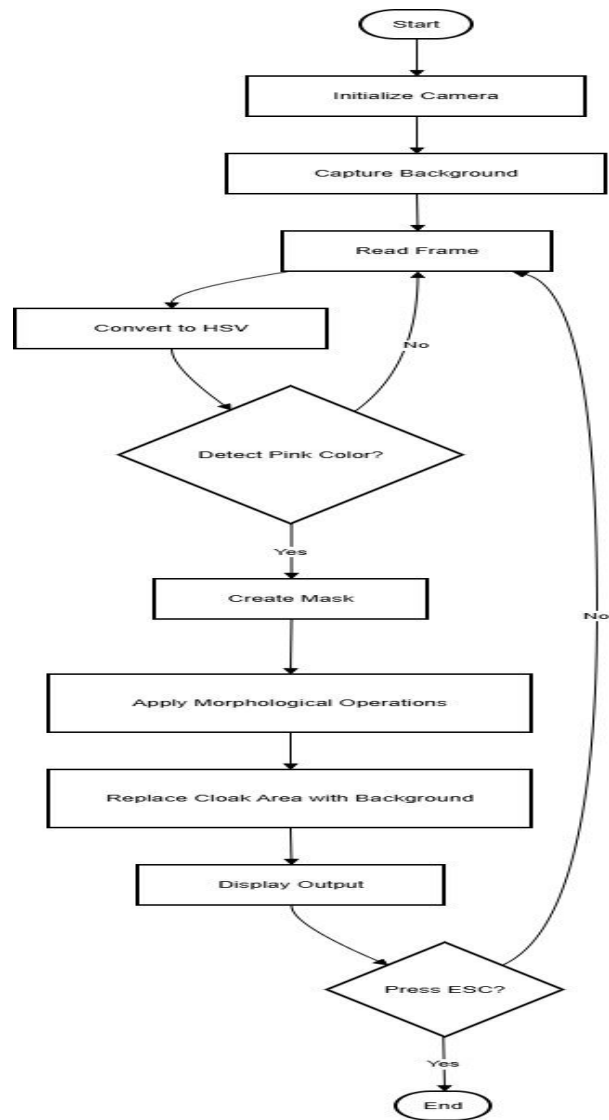
Name	Description
Frame	Image captured from webcam
Background	Stored initial frame
Mask	Binary image for color detection
Output	Binary image for color detection

3.2.1 Normalization

Not applicable since no database is used.

3.3 Component Design

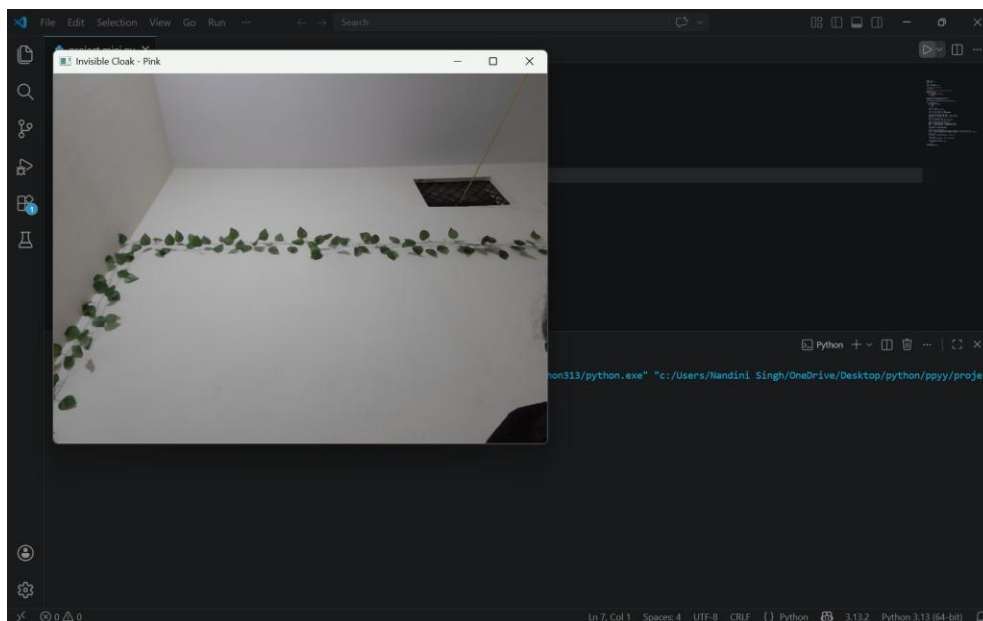
3.3.1 Flowchart

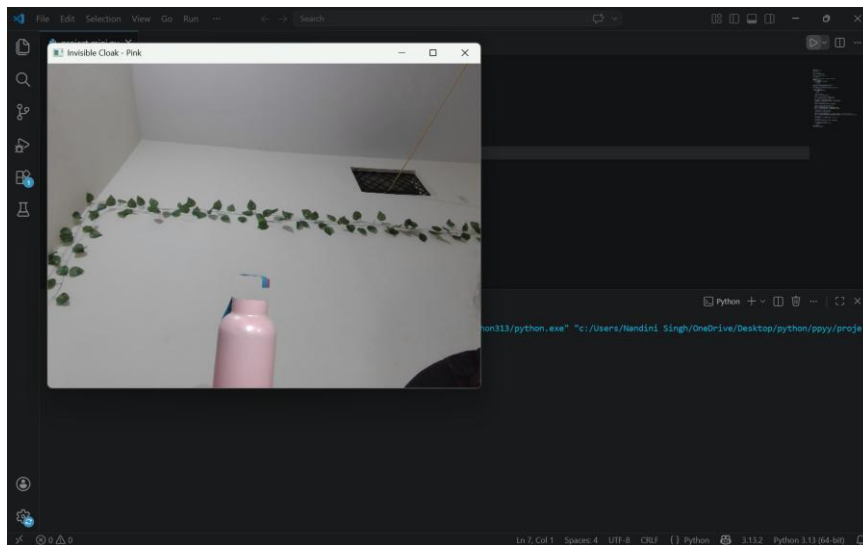


3.4 Interface Design

3.4.1 Screen Shots


```
File Edit Selection View Go Run ... Search
project mini.py X
C: > Users > Nandini Singh > OneDrive > Desktop > python > ppyy > project mini.py > ...
1 import cv2
2 import numpy as np
3 import time
4
5 # Open the webcam
6 cap = cv2.VideoCapture(0)
7
8 # Allow the camera to warm up
9 time.sleep(3)
10
11 # Capture the background (30 frames for stability)
12 background = 0
13 for i in range(30):
14     ret, background = cap.read()
15     if not ret:
16         continue
17
18 # Flip the background (to match mirror view)
19 background = np.flip(background, axis=1)
20
21 print("Background captured! Now wear your pink cloth...")
22
23 while cap.isOpened():
24     ret, frame = cap.read()
25     if not ret:
26         break
27
28     # Flip the frame
29     frame = np.flip(frame, axis=1)
30
31     # Convert frame to HSV color space
32     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
33
34     # Define HSV range for pink cloak
35     lower_pink = np.array([140, 80, 80]) # lower HSV bound
36     upper_pink = np.array([170, 255, 255]) # upper HSV bound
```





Appendix A: Sample Code Snippet

```
import cv2

import numpy as np

cap = cv2.VideoCapture(0)

# Capture background

for i in range(30):

    ret, background = cap.read()

while True:

    ret, frame = cap.read()

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    lower_pink = np.array([140, 50, 50])

    upper_pink = np.array([170, 255, 255])

    mask = cv2.inRange(hsv, lower_pink, upper_pink)

    result = np.where(mask[:, :, None] == 255, background, frame)

    cv2.imshow("Invisible Cloak", result)

    if cv2.waitKey(1) == 27:

        break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Appendix B: Installation Commands

```
pip install opencv-python
```

```
pip install numpy
```